

INTENSIVE BOOTCAMP MANUAL

FULL STACK WEB DEVELOPER

NATASSHA P. CUA

08/2026

INTRODUCTION

This manual is designed for three kinds of learners: complete beginners who have never written code, career shifters who need a structured path into software development, and refreshers who have seen some web technologies before but want to rebuild confidence and connect the pieces into a full-stack workflow.

For beginners

Read the explanations in order, type every code example yourself, and do not worry about memorizing syntax. Your first goal is to recognize patterns and understand why each part exists.

For career shifters

Connect each technical concept to a workplace responsibility: building interfaces, collaborating through Git, designing APIs, debugging systems, documenting work, and communicating technical decisions.

For refreshers

Use the chapter summaries and checkpoints to identify weak areas. Rebuild the labs without copying the examples, then compare your solution with the reference patterns.

BOOTCAMP LEARNING PHILOSOPHY

Full-stack development is not a collection of disconnected tools. It is a flow: a user interacts with an interface; the browser runs front-end code; the front end sends an HTTP request; a server validates and processes that request; a database stores or retrieves information; the server returns a response; and the interface updates. Every stage of this bootcamp adds one more piece to that flow.

- Understand before memorizing: focus on concepts, then syntax.
- Code along: reading code is not the same as writing and debugging it.
- Use small experiments: change one thing, observe the result, then explain why it changed.
- Save working milestones with Git so you can recover and compare versions.
- Ask specific questions: describe what you expected, what happened, and what you already tried.
- Treat errors as evidence. Error messages, logs, network responses, and test results are clues.

15 SESSION ROADMAP

Session	Focus	Core outcome
1	Developer Foundations	Web basics, clients/servers, HTTP, DevTools, terminal
2	Development Workflow	VS Code, Git, GitHub, Markdown, README
3	Semantic HTML	Structure, accessibility, forms, multipage organization
4	CSS Fundamentals	Selectors, cascade, box model, Flexbox, debugging
5	Responsive Design	Mobile-first, media queries, Bootstrap, Capstone 1
6	JavaScript Fundamentals	Variables, types, conditions, loops, functions
7	Modern JavaScript	Arrays, objects, methods, ES6, destructuring
8	Browser JavaScript	DOM, events, forms, JSON, localStorage, fetch
9	Node & Express	npm, servers, routes, middleware, environment variables
10	MongoDB & Mongoose	Documents, schemas, models, CRUD, validation
11	REST API Engineering	Endpoints, HTTP status codes, Postman, Capstone 2
12	React Foundations	JSX, components, props, state, lists, events
13	React Full Stack	Router, forms, effects, API integration, Capstone 3
14	Capstone Completion	Testing, debugging, documentation, deployment
15	Career Launch	Demo day, GitHub, résumé, interviews, growth plan

TABLE OF CONTENTS

INTRODUCTION

- For Complete Beginners
- For Career Shifters
- For Refreshers
- Bootcamp Learning Philosophy
- 15-Session Bootcamp Roadmap
- Standard 4-Hour Session Rhythm

Stage 1 — Developer Foundations

Session 1 — Understanding Web Development

- 1.1 What Web Development Actually Is
- 1.2 Front-End, Back-End, and Full-Stack Roles
- 1.3 How the Web Works
- 1.4 Internet, Browser, HTTP, Domain, Hosting, and Deployment
- 1.5 Browser Developer Tools
- 1.6 Terminal Basics — Windows and macOS
- 1.7 Creating Your First index.html
- 1.8 Responsible Use of Online Resources and AI
- 1.9 Guided Lab — Create the Bootcamp Workspace
- 1.10 Common Beginner Mistakes
- 1.11 Check Your Understanding

Session 2 — VS Code, Git, and GitHub Workflow

- 2.1 VS Code as Your Development Workspace
- 2.2 Recommended VS Code Extensions
- 2.3 Why Version Control Exists
- 2.4 The Git Mental Model
- 2.5 Understanding Core Git Commands
- 2.6 Git Identity and Authentication
- 2.7 Branches and Project History
- 2.8 Local and Remote Repositories
- 2.9 Markdown and README Documentation
- 2.10 Guided Lab — First GitHub Workflow
- 2.11 Common Beginner Mistakes
- 2.12 Check Your Understanding

Stage 2 — Front-End Craft

Session 3 — Semantic HTML and Accessible Page Structure

- 3.1 HTML Is Structure, Not Decoration
- 3.2 Core Document Structure
- 3.3 Text, Links, Images, and Lists
- 3.4 Relative and Absolute Paths
- 3.5 Forms and Labels
- 3.6 Accessibility Fundamentals
- 3.7 Guided Lab — Semantic Personal Profile
- 3.8 Common Beginner Mistakes
- 3.9 Check Your Understanding

Session 4 — CSS Fundamentals and Page Layout

- 4.1 What CSS Does
- 4.2 Selectors, Cascade, Specificity, and Inheritance
- 4.3 The CSS Box Model
- 4.4 CSS Units and Responsive-Friendly Sizing
- 4.5 Flexbox Layout
- 4.6 Debugging CSS with Browser DevTools
- 4.7 Guided Lab — Style the Portfolio
- 4.8 Common Beginner Mistakes
- 4.9 Check Your Understanding

Session 5 — Responsive Design, Bootstrap, and Capstone 1

- 5.1 Responsive Design as a Behavior
- 5.2 Mobile-First Development
- 5.3 Bootstrap as a Design System and Utility Toolkit
- 5.4 Responsive Testing
- 5.5 Capstone 1 — Personal Web Portfolio
- 5.6 Capstone 1 Completion Checklist
- 5.7 Common Beginner Mistakes
- 5.8 Check Your Understanding

Stage 3 — JavaScript Essentials

Session 6 — JavaScript Fundamentals

- 6.1 How JavaScript Changes a Webpage into a Program
- 6.2 Values and Data Types
- 6.3 Variables — let and const
- 6.4 Operators and Comparisons
- 6.5 Conditional Statements
- 6.6 Loops and Repetition
- 6.7 Functions, Parameters, and Return Values
- 6.8 Scope and Debugging
- 6.9 Guided Lab — Learner Score Evaluator
- 6.10 Common Beginner Mistakes
- 6.11 Check Your Understanding

Session 7 — Arrays, Objects, and Modern JavaScript

- 7.1 Modeling Application Data with Arrays and Objects
- 7.2 Core Array Methods
- 7.3 Object Access and Destructuring
- 7.4 Spread Syntax and Immutability Habits
- 7.5 Arrow Functions and Callbacks
- 7.6 Template Literals
- 7.7 Guided Lab — Product Data Explorer
- 7.8 Common Beginner Mistakes
- 7.9 Check Your Understanding

Session 8 — DOM Manipulation, Events, JSON, and Browser Storage

- 8.1 The DOM and the Browser
- 8.2 Selecting and Modifying Elements
- 8.3 Events and Event Objects
- 8.4 Working with Forms and User Input
- 8.5 Understanding JSON
- 8.6 Using localStorage
- 8.7 Introduction to fetch and Asynchronous Requests
- 8.8 Guided Lab — Persistent Task Manager
- 8.9 Common Beginner Mistakes
- 8.10 Check Your Understanding

Stage 4 — Back-End Engineering

Session 9 — Node.js, npm, Express, and Server Fundamentals

- 9.1 What Node.js Changes
- 9.2 npm and package.json
- 9.3 Creating an Express Server
- 9.4 Routes and Route Parameters
- 9.5 Understanding Middleware
- 9.6 Environment Variables and Configuration
- 9.7 Error Handling and 404 Responses
- 9.8 Guided Lab — First Express API
- 9.9 Common Beginner Mistakes
- 9.10 Check Your Understanding

Session 10 — MongoDB, Mongoose, and Database Operations

- 10.1 Why Databases Exist
- 10.2 The MongoDB Document Model
- 10.3 Mongoose Schemas and Models
- 10.4 Connecting Node.js to MongoDB
- 10.5 CRUD Operations
- 10.6 Validation Across Application Layers
- 10.7 Guided Lab — Persistent Product Model
- 10.8 Common Beginner Mistakes
- 10.9 Check Your Understanding

Session 11 — REST API Design, Postman Testing, and Capstone 2

- 11.1 REST as a Resource-Oriented Style
- 11.2 HTTP Status Codes as Part of the API Contract
- 11.3 Request Parameters, Query Strings, Bodies, and Headers
- 11.4 Separating Routes, Controllers, Models, and Middleware
- 11.5 Postman as a Repeatable API Test Client
- 11.6 Authentication and Authorization Concepts
- 11.7 Capstone 2 — E-Commerce REST API
- 11.8 Capstone 2 Quality Gate
- 11.9 Common Beginner Mistakes
- 11.10 Check Your Understanding

Stage 5 — Full-Stack Development with React

Session 12 - React Fundamentals: Components, Props, and State

- 12.1 Why React?
- 12.2 Understanding JSX
- 12.3 Components and Composition
- 12.4 Props
- 12.5 State with useState
- 12.6 Rendering Lists and Using Keys
- 12.7 Conditional Rendering
- 12.8 Guided Lab — React Product Gallery
- 12.9 Common Beginner Mistakes
- 12.10 Check Your Understanding

Session 13 — React Forms, Routing, Effects, and API Integration

- 13.1 Controlled React Forms
- 13.2 Routing with React Router
- 13.3 useEffect and External Synchronization
- 13.4 Loading, Error, Empty, and Success States
- 13.5 Sending Data to the API
- 13.6 CORS and Environment Configuration
- 13.7 Capstone 3 — React Full-Stack Application
- 13.8 Guided Lab — Connect React to the REST API
- 13.9 Common Beginner Mistakes
- 13.10 Check Your Understanding

Stage 6 — Capstone Project Completion

Session 14 — Testing, Debugging, Documentation, and Deployment

- 14.1 Debugging as a Systematic Process
- 14.2 Full-Stack Debugging Across Application Layers
- 14.3 Testing Categories for Bootcamp Projects
- 14.4 Refactoring Safely
- 14.5 README as Operational Documentation
- 14.6 Deployment Concepts
- 14.7 Security Hygiene Before Publishing
- 14.8 Capstone Audit Sprint
- 14.9 Common Beginner Mistakes
- 14.10 Check Your Understanding

Stage 7 — Commencement and Career Launch

Session 15 — Demo Day, GitHub Portfolio, and Career Launch

- 15.1 What a Strong Project Presentation Proves
- 15.2 Presenting the Problem, Users, Features, and Architecture
- 15.3 Demonstrating Technical Challenges and Solutions
- 15.4 GitHub as Professional Evidence
- 15.5 Turning Projects into Résumé Bullet Points
- 15.6 Technical Interview Mindset
- 15.7 Preparing for Coding Assessments
- 15.8 Career Paths After the Bootcamp
- 15.9 Building a 90-Day Continuation Plan
- 15.10 Final Demo Checklist
- 15.11 Common Career-Launch Mistakes
- 15.12 Check Your Understanding

Appendices

Appendix A — Windows and macOS Command Quick Reference

- Terminal Navigation
- File and Folder Commands
- VS Code Command-Line Usage
- Git Verification
- Node.js and npm Verification

Appendix B — Git Cheat Sheet

- Repository Setup
- Status and Differences
- Staging and Committing
- Branches
- Remotes
- Push and Pull

Appendix C — HTTP and REST Quick Reference

- HTTP Methods
- Common Status Codes
- Requests and Responses
- REST Resource Patterns

Appendix D — HTML and CSS Starter Reference

- Basic HTML Document
- Semantic Layout
- CSS Reset / Base Styles
- Responsive Container Pattern

Appendix E — JavaScript Starter Reference

- Variables and Functions
- Arrays and Array Methods
- DOM Events
- Basic Data Manipulation

Appendix F — Node.js, Express, and Mongoose Starter Structure

- Recommended Project Folder Structure
- Routes
- Controllers
- Models
- Middleware
- Environment Configuration

Appendix G — React Mental Model

- Components
- Props
- State
- Events
- Derived Data
- Effects
- Routing
- API Integration

Appendix H — Capstone Quality Checklist

- Functionality
- Code Quality
- Responsive Design
- Accessibility
- Git and GitHub
- README Documentation
- REST API Quality

- Database Quality
- React Quality
- Deployment Readiness

Appendix I — Glossary for New Developers

- API
- Branch
- Browser
- Build
- Client
- Commit
- Component
- CORS
- CRUD
- Database
- Dependency
- Deployment
- DNS
- DOM
- Endpoint
- Environment Variable
- HTTP
- JSON
- Middleware
- Model
- npm
- Props
- Repository
- Request
- Response
- Route
- Server
- State
- Validation
- Version Control

Appendix J — Independent Practice Ladder

1. Rebuild a Static HTML Page
2. Add Responsive CSS
3. Add JavaScript Interaction
4. Add Browser Storage
5. Build an Express API

6. Add MongoDB Persistence
7. Test with Postman
8. Connect a React Client
9. Deploy the Full Stack
10. Explain the Complete Request Flow

Further References / Readings

.

SESSION 01

UNDERSTANDING WEB DEVELOPMENT

Stage 1 — Developer Foundations

Learning objectives

- ✓ Explain the difference between front-end, back-end, and full-stack development.
- ✓ Describe how a browser, server, HTTP request/response, and database work together.
- ✓ Define browser, internet, HTTP/HTTPS, domain, hosting, and deployment in practical terms.
- ✓ Inspect a webpage using browser Developer Tools.
- ✓ Use basic terminal commands to create and navigate a project folder.
- ✓ Create and open a first index.html page.
- ✓ Use online resources and AI coding tools responsibly.

1.1 What web development actually is

Web development is the practice of building software that people access through web technologies. A simple informational page, an online shop, a banking portal, a student system, and a browser-based project tracker all use the web platform, but they differ in how much interaction, data processing, and server-side logic they require.

A useful distinction is between a website and a web application. A website often emphasizes publishing and navigation: readers consume articles, product information, contact details, or documentation. A web application behaves more like software: users sign in, create records, upload data, search, filter, purchase, message, or trigger workflows. Modern products frequently combine both.

1.2 Front-end, back-end, and full-stack roles

The **front end** is the part that runs in the user's browser and is directly experienced by the user. HTML describes structure, CSS controls presentation and layout, and JavaScript adds behavior. Later in the bootcamp, React will help you organize larger interactive interfaces into reusable components.

The **back end** runs on a server. It receives requests, enforces rules, validates input, reads and writes data, manages authentication, integrates with other services, and returns responses. In

this bootcamp, Node.js runs JavaScript on the server and Express helps define routes and middleware.

A **full-stack developer** can work across the boundary between interface and server. This does not mean knowing every tool in the industry. It means understanding the complete request flow well enough to design, build, debug, and communicate across layers.

Mental model

HTML = structure. CSS = presentation.
JavaScript = behavior. Node/Express = server logic. MongoDB = persistent data. React = organized interactive UI. Git/GitHub = change history and collaboration.

1.3 How the web works

When a user enters a web address, the browser must locate the destination, establish a connection, send an HTTP request, receive a response, and render the returned resources. The real internet is more complex, but this simplified model is enough to understand most application debugging.

1. The user enters a URL or clicks a link.
2. The browser uses DNS to discover the network address associated with the domain.
3. The browser connects to the server, normally using encrypted HTTPS.
4. The browser sends an HTTP request such as GET /products/123.
5. The server routes the request to application code.
6. The server may query a database or another service.
7. The server creates an HTTP response with a status code, headers, and content/data.
8. The browser receives the response and updates what the user sees.

```
Browser / React -> HTTP Request -> Node + Express -> MongoDB  
Browser / React <- HTTP Response <- Node + Express <- MongoDB
```

1.4 Internet, browser, HTTP, domain, hosting, deployment

Term	Beginner-friendly meaning	Example
Internet	The global network that connects devices and networks.	Your laptop reaching a cloud server.
Browser	Software that retrieves and runs web content.	Chrome, Edge, Firefox, Safari.
HTTP	Rules for web requests and responses.	GET /products
HTTPS	HTTP protected by encryption in transit.	https://example.com
Domain	Human-readable name used to reach a service.	example.com
Hosting	Infrastructure that keeps your app available to users.	A cloud hosting platform.
Deployment	Moving a tested application into a target environment.	Publishing your portfolio online.

HTTP methods communicate intent. GET typically retrieves, POST creates or performs an action, PUT/PATCH updates, and DELETE removes. You will use these heavily when building APIs. Status codes summarize the result: 2xx means success, 4xx means the client request has a problem, and 5xx means the server failed to complete a valid request.

1.5 Browser Developer Tools

Developer Tools are built into modern browsers. They let you inspect the page that the browser is actually rendering, view CSS rules, run JavaScript, watch network requests, and investigate errors. Developers use DevTools constantly because it reveals what the browser knows rather than what we assume the browser knows.

- Elements: inspect HTML structure and temporarily edit page content.
- Styles/Computed: inspect which CSS rules win and where they came from.
- Console: view JavaScript errors and run quick experiments.
- Network: inspect requests, status codes, payloads, response bodies, and timing.
- Application/Storage: inspect localStorage, cookies, and other browser-side storage.

1.6 Terminal basics — Windows and macOS

Task	Windows PowerShell	macOS Terminal
Show current folder	pwd	pwd
List files/folders	ls	ls
Move into folder	cd folder-name	cd folder-name
Go up one level	cd ..	cd ..
Create folder	mkdir folder-name	mkdir folder-name
Go to home folder	cd ~	cd ~
Clear terminal	cls	clear
Open current folder in VS Code	code .	code . (after enabling VS Code shell command)

Why learn the terminal?

Modern tooling is command-driven. Git, npm, Node.js, Vite, test runners, deployment tools, and many cloud workflows are easiest to understand when the terminal is not intimidating.

1.7 Your first index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Your Name</h1>
  <h2>Aspiring Full-Stack Web Developer</h2>
  <p>I am learning how the web works and how to build web applications.</p>

  <h2>My Learning Goals</h2>
  <ul>
    <li>Build responsive websites</li>
    <li>Learn JavaScript and APIs</li>
    <li>Build a full-stack project</li>
  </ul>
</body>
</html>
```

The file name `index.html` is a common convention for the default page of a directory. The browser parses the HTML from top to bottom, builds an internal tree called the DOM, and then renders the visible content. For now, focus on recognizing opening tags, closing tags, nesting, and the difference between the head (document metadata/configuration) and body (visible document content).

Guided lab: create the bootcamp workspace

1. Create a folder named mstconnect-fullstack-bootcamp inside Documents.
2. Inside it, create session-01, exercises, and projects.
3. Open session-01 in VS Code.
4. Create index.html and type the sample document.
5. Save the file, open it in a browser, then use Inspect to locate the h1 element.
6. Personalize the page with your name, a short introduction, and three goals.

Expected result

A working local webpage that you can explain line by line.

1.8 Responsible use of online resource and AI

Developers constantly consult documentation, examples, forums, search engines, and AI assistants. The skill is not avoiding help; it is using help without surrendering understanding. Treat generated suggestions as hypotheses to test, not facts to trust automatically.

- Ask for explanations, hints, examples, or debugging steps before asking for complete solutions.
- Never submit code you cannot explain.
- Test generated code and read errors instead of repeatedly prompting for random fixes.
- Do not paste passwords, API keys, database credentials, private client data, or confidential information into public tools.
- Prefer official documentation when behavior or configuration matters.

Common beginner mistakes

Trying to memorize everything on day one — Learn the mental model first. Syntax becomes easier after repeated use.

Double-clicking random files instead of opening the project folder in VS Code — Open the project root so file paths and terminal commands make sense.

Ignoring file extensions — index.html is different from index.html.txt. Make extensions visible when troubleshooting.

Using AI to generate an entire assignment — Use AI to explain and review; keep yourself responsible for design, testing, and understanding.

Check your understanding

1. What is the role of the browser in the client/server model?
2. What is the difference between HTTP and HTTPS?
3. What belongs on the front end, and what usually belongs on the back end?
4. Why is a database different from a JavaScript variable?
5. What does the Network panel in DevTools help you inspect?

SESSION 02

VS Code, Git, and GitHub Workflow

Stage 1 — Developer Foundations

Learning objectives

- ✓ Configure VS Code for a consistent beginner-friendly workflow.
- ✓ Explain why version control matters and how Git stores project history.
- ✓ Initialize a repository, stage changes, commit, and inspect repository status.
- ✓ Explain working tree, staging area, local repository, and remote repository.
- ✓ Create a GitHub repository, connect it to a local repository, and push commits.
- ✓ Write basic Markdown and a useful README.

2.1 VS Code as your development workspace

Visual Studio Code is an editor rather than a full operating environment. It becomes powerful because it understands project folders, provides syntax highlighting and extensions, includes a terminal, and integrates with source control. The most important habit is to open the project folder, not just an individual file. That gives VS Code context about related files, relative paths, Git status, and workspace settings.

- Explorer: create, rename, move, and inspect project files.
- Editor tabs: work across multiple files without losing project context.
- Integrated terminal: run Git, Node, npm, and other commands in the project folder.
- Source Control view: inspect file changes and commits visually.
- Extensions: add language support, formatting, linting, and workflow helpers.

2.2 Recommended extensions

Extensions should reduce friction, not hide fundamentals. Avoid installing dozens at once. A small stable set is easier to troubleshoot.

Extension type	Purpose	Use carefully because...
Prettier	Consistent formatting	Formatting does not fix incorrect logic.
ESLint	Flags JavaScript problems/style issues	Rules may differ across projects.
Live Server or similar	Convenient static-page development	Do not confuse it with a production server.
GitLens or Git helpers	Extra history/blame context	Learn core Git commands first.
Language/framework extensions	Better completion/snippets	Snippets should not replace understanding.

2.3 Why version control exists

Without version control, developers often create files named final, final-final, final-v2, and final-really-final. Git replaces that fragile pattern with a structured history. A commit is a named snapshot of tracked changes plus metadata such as author and time. Because commits form history, you can compare versions, investigate when a bug appeared, restore earlier work, and collaborate without emailing project folders back and forth.

Git vs GitHub

Git is the version-control system that records history locally. GitHub is an online platform that hosts Git repositories and adds collaboration features such as pull requests, issues, reviews, and project visibility.

2.4 The Git mental model

Working files --git add--> Staging area --git commit--> Local repository --git push-->
Remote repository

The working tree is what you are editing right now. The staging area is your proposed next commit. The local repository stores commits on your machine. A remote repository is another copy of the project, commonly hosted on GitHub. Keeping these concepts separate prevents many beginner mistakes.

```
git init
git status
git add index.html
git status
git commit -m "Create first bootcamp page"
git log --oneline
```

2.5 What the core commands mean

Command	Meaning
git init	Start Git tracking in the current folder.
git status	Show changed, staged, and untracked files.
git add <file>	Stage a file for the next commit.
git add .	Stage changes under the current directory; inspect status first.
git commit -m "message"	Create a local commit from staged changes.
git log --oneline	View compact commit history.
git diff	View unstaged changes.
git diff --staged	View changes currently staged.

2.6 Git identity and authentication

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global -list
```

Git records author information in commits. This is not your password. Authentication to GitHub is separate and may use browser sign-in, credential managers, or SSH depending on the setup. Follow the instructor's chosen method so the class uses one consistent workflow.

2.7 Branches and history

A branch is a movable name pointing to a line of commits. Branches let developers work on features or experiments without immediately changing the main line. At this stage, understand the concept even if most exercises remain on the main branch. Later, branching becomes important for collaboration and safe changes.

```
git branch
git switch -c feature/about-section
# edit files
git add .
git commit -m "Add about section"
git switch main
```

2.8 Local and remote repositories

A remote is simply a named location for another Git repository. The conventional name origin is used for the remote you cloned from or first connected to. Pushing sends commits that already exist locally. It does not upload uncommitted work.

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git branch -M main
git push -u origin main
```

Important

If git push says “Everything up-to-date” but your latest edits are missing on GitHub, check git status. The likely issue is that the changes were never committed.

2.9 Markdown and README documentation

Markdown is a lightweight syntax for formatted text. GitHub renders README.md automatically on repository pages. A good README explains what the project is, why it exists, how to run it, what technologies it uses, and what the learner practiced. Documentation is part of software quality because code that cannot be understood or started by someone else is difficult to maintain.

```
# My First Bootcamp Project

## About
This repository contains my Session 1 HTML exercise.

## What I Practiced
- HTML document structure
- Terminal navigation
- Git commits

## How to Run
Open `index.html` in a browser.

## Author
Your Name
```

Guided lab: first GitHub workflow

1. Open the Session 1 project folder in VS Code.
2. Verify Git is installed with `git --version`.
3. Run `git init` and inspect git status.
4. Create README.md and describe the project.
5. Stage the files, then create the first commit.
6. Create an empty GitHub repository.
7. Add the GitHub repository as origin and push main.
8. Edit one line in index.html, inspect git diff, commit the change, and push again.

Expected result

A GitHub repository containing index.html, README.md, and at least two meaningful commits.

Common beginner mistakes

Running git init in the wrong folder — Use `pwd` and `ls` first. Initialize the project root, not Documents or your entire home folder.

Committing generated secrets or credentials — Use `.gitignore` and environment variables later in the bootcamp.

Using meaningless messages such as “update” — Describe the change: “Add navigation links” is more useful.

Thinking push saves current edits automatically — Git only pushes commits. Stage and commit first.

Check your understanding

1. What is the difference between the working tree and staging area?
2. Why might you create several small commits instead of one giant commit?
3. What is the difference between Git and GitHub?
4. What does origin represent?
5. What information should a README provide?

SESSION 03

Semantic HTML and Accessible Page Structure

Stage 2 — Front-End Craft

Learning objectives

- ✓ Build a valid semantic HTML document using meaningful structural elements.
- ✓ Create links, images, lists, tables, and forms using appropriate elements and attributes.
- ✓ Explain why semantic structure improves accessibility, maintainability, and search understanding.
- ✓ Use labels and input types to make forms easier to use.
- ✓ Organize a simple multipage site with sensible file paths.

3.1 HTML is structure, not decoration

HTML describes the meaning and hierarchy of content. A heading is not just “large text”; it labels a section. A nav element is not just a container; it identifies major navigation. Semantic HTML allows browsers, assistive technologies, search engines, and future developers to understand the document more accurately.

```
<header>
  <h1>My Portfolio</h1>
  <nav aria-label="Primary">
    <a href="#about">About</a>
    <a href="#projects">Projects</a>
    <a href="#contact">Contact</a>
  </nav>
</header>

<main>
  <section id="about">...</section>
  <section id="projects">...</section>
</main>

<footer>...</footer>
```


3.2 Core document structure

Element	Purpose
<header>	Introductory content for a page or section.
<nav>	Major navigation links.
<main>	Primary unique content of the page.
<section>	Thematic grouping, normally with a heading.
<article>	Self-contained content that can stand independently.
<aside>	Related but secondary content.
<footer>	Closing information for a page or section.

3.3 Text, links, images, and lists

Choose elements by meaning. Use headings to create an outline, paragraphs for prose, lists when order or grouping matters, and links for navigation. Images require a source and usually alternative text. Alternative text should communicate the purpose of the image in context rather than mechanically describe every pixel.

```
<h2>Projects</h2>
<ul>
  <li>Portfolio website</li>
  <li>E-commerce API</li>
  <li>React application</li>
</ul>

<a href="https://github.com/yourname">Visit my GitHub</a>


```

3.4 Relative and absolute paths

An absolute URL includes the full scheme and domain, such as `https://example.com/docs`. A relative path is resolved from the current document location, such as `images/logo.png` or `../index.html`. File-path problems are common because browsers follow exact directory relationships; capitalization can also matter on servers even if your local operating system appears forgiving.

```
portfolio/
├─ index.html
├─ about.html
├─ images/
│   └─ profile.jpg
└─ css/
    └─ style.css
```

3.5 Forms and labels

Forms collect information. A good form pairs each control with a meaningful label, uses the correct input type, explains required formats, and provides clear feedback. Client-side validation improves usability, but server-side validation is still required because browsers cannot be trusted to enforce business rules for a server.

```
<form>
  <div>
    <label for="name">Name</label>
    <input id="name" name="name" type="text" required>
  </div>

  <div>
    <label for="email">Email</label>
    <input id="email" name="email" type="email" required>
  </div>

  <button type="submit">Send message</button>
</form>
```

3.6 Accessibility fundamentals

Accessibility means designing so people with different abilities, devices, and interaction methods can use the product. Keyboard navigation, sufficient color contrast, logical heading order, meaningful link text, visible focus, labels, and semantic controls help many users and also improve overall usability.

- Use real buttons for actions and real links for navigation.
- Do not rely only on color to communicate state.
- Keep headings in a logical hierarchy.
- Provide labels for inputs and useful alternative text for informative images.
- Make sure keyboard focus is visible and interactive elements are reachable.
- Use ARIA only when native HTML cannot express the needed semantics.

Guided lab: semantic personal profile

1. Refactor your Session 1 page into header, nav, main, sections, and footer.
2. Add About, Skills, Projects, and Contact sections.
3. Add at least one image with meaningful alt text.
4. Add a labeled contact form with name, email, message, and submit button.
5. Test internal navigation links and keyboard tab order.
6. Commit and push the changes to GitHub.

Expected result

A semantic, navigable HTML portfolio structure ready for CSS styling.

Common beginner mistakes

Using div for everything — Use semantic elements when they accurately describe the content.

Skipping labels because placeholders “look enough” — Placeholders disappear and are not a reliable replacement for labels.

Using headings based on size — Choose heading levels based on document structure, then style them with CSS.

Broken image paths — Check folder names, file names, relative paths, and capitalization.

Check your understanding

1. Why is semantic HTML more than a style preference?
2. When should you use a button instead of a link?
3. What makes alternative text useful?
4. Why does client-side form validation not replace server validation?
5. How do relative paths depend on the current file location?

SESSION 04

CSS Fundamentals and Page Layout

Stage 2 — Front-End Craft

Learning objectives

- ✓ Explain the cascade, inheritance, specificity, and source order.
- ✓ Apply typography, color, spacing, borders, and backgrounds with reusable classes.
- ✓ Use the CSS box model to reason about element size and spacing.
- ✓ Build common layouts with Flexbox.
- ✓ Use browser DevTools to debug computed styles and layout problems.

4.1 What CSS does

CSS is a rule system that matches elements and assigns presentation. The browser collects rules from several sources, resolves conflicts through the cascade, computes final values, performs layout, then paints the page. Understanding that sequence is more useful than memorizing hundreds of properties.

```
/* selector { property: value; } */  
body {  
  font-family: system-ui, sans-serif;  
  color: #1f2937;  
  background: #ffffff;  
}  
  
.card {  
  padding: 1rem;  
  border: 1px solid #d1d5db;  
  border-radius: 0.75rem;  
}
```

4.2 Selectors, cascade, specificity, and inheritance

A selector identifies elements. When several rules target the same property, the browser resolves the conflict using importance, origin, specificity, and source order. Inheritance allows some properties, especially text-related ones, to flow from parent to child. Avoid fighting the cascade with excessive !important rules; prefer clear structure and predictable selectors.

Selector	Example	Use
Element	p	Broad element defaults.
Class	.card	Reusable component styling.
ID	#contact	Unique hooks; usually avoid for general styling.
Descendant	.nav a	Elements inside a context.
Pseudo-class	button:hover	State or structural condition.

4.3 The box model

Every visible element occupies a rectangular box made of content, padding, border, and margin. Width/height calculations become easier when you set box-sizing: border-box globally, because declared width then includes padding and border.

```
*, *::before, *::after {
  box-sizing: border-box;
}

.card {
  width: 320px;
  padding: 24px;
  border: 1px solid #ccc;
  margin: 16px;
}
```

4.4 Units and responsive-friendly sizing

Pixels are useful for precise borders and small details, but flexible interfaces also use rem for scalable typography/spacing, percentages for relative widths, and viewport units for layout effects. Do not chase one “perfect” unit. Choose units based on what should remain fixed and what should adapt.

4.5 Flexbox

Flexbox lays out items along one main axis and one cross axis. It is ideal for navigation bars, card rows, toolbars, centered blocks, and many one-dimensional arrangements. First decide the main direction, then control distribution and alignment.

```
.nav {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 1rem;  
}  
  
.project-list {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1.25rem;  
}  
  
.project-card {  
  flex: 1 1 280px;  
}
```

4.6 Debugging CSS with DevTools

When a style is not working, inspect the element rather than guessing. DevTools shows matched rules, crossed-out declarations, computed values, box-model dimensions, and layout overlays.

Ask: Did my selector match? Was the property overridden? Is the value invalid? Is a parent layout controlling the child?

Guided lab: style the portfolio

1. Create `css/style.css` and link it from `index.html`.
2. Add a global box-sizing rule and base typography.
3. Style the page header, navigation, sections, and buttons.
4. Create reusable `.card` and `.container` classes.
5. Build the Projects section using Flexbox.
6. Use DevTools to inspect spacing and identify one overridden rule.
7. Commit the styling milestone to Git.

Expected result

A readable, consistent portfolio that uses external CSS and Flexbox.

Common beginner mistakes

Editing CSS but the page does not change — Check the <link> path, save files, and inspect whether the rule is loaded.

Using margin for every spacing problem — Decide whether the space belongs inside the component (padding), between siblings (gap), or outside the box (margin).

Overusing !important — Fix selector design and cascade order instead of forcing every rule.

Setting fixed widths everywhere — Prefer flexible widths, max-width, and wrapping for responsive layouts.

Check your understanding

1. What four regions make up the box model?
2. What does box-sizing: border-box change?
3. How does a class selector differ from an element selector?
4. What is the difference between justify-content and align-items in Flexbox?
5. How can DevTools tell you why a declaration is not taking effect?

SESSION 05

Responsive Design, Bootstrap, and Capstone 1

Stage 2 — Front-End Craft

Learning objectives

- ✓ Apply mobile-first responsive design principles.
- ✓ Use media queries and flexible layouts to support multiple screen sizes.
- ✓ Use Bootstrap containers, grid, utilities, and components intentionally.
- ✓ Test a site using responsive browser tools and keyboard navigation.
- ✓ Complete and publish Capstone 1: Personal Web Portfolio.

5.1 Responsive design is a behavior, not a device list

Responsive design means the interface adapts to available space and user needs. Rather than building separate “phone” and “desktop” sites, start with a flexible base and introduce layout changes when the content needs them. Breakpoints are decisions about design behavior, not labels for specific device brands.

```
/* Mobile-first base */
.project-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}

@media (min-width: 768px) {
  .project-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1100px) {
  .project-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}
```


5.2 Mobile-first thinking

Mobile-first means the default CSS supports narrow viewports, then larger-screen enhancements are added with min-width media queries. This often encourages cleaner priorities: essential content first, simpler navigation, touch-friendly controls, and fewer assumptions about available space.

5.3 Bootstrap as a design system and utility toolkit

Bootstrap provides prewritten CSS classes and JavaScript-enhanced components. It can accelerate consistent layouts, especially during a bootcamp, but it does not replace CSS knowledge. You should be able to explain what a container, row, column, spacing utility, breakpoint class, and component are doing.

```
<div class="container py-5">
  <div class="row g-4">
    <div class="col-12 col-md-6 col-lg-4">
      <article class="card h-100">
        <div class="card-body">
          <h2 class="h5">Project One</h2>
          <p class="card-text">Responsive portfolio project.</p>
        </div>
      </article>
    </div>
  </div>
</div>
```

5.4 Responsive testing

Testing should include narrow and wide windows, zoom, long content, keyboard navigation, and real devices when available. A page that “looks okay” at one screenshot width may still fail when text wraps, users zoom to 200%, or a button receives keyboard focus.

- Resize gradually instead of testing only three preset widths.
- Check that no horizontal scrolling appears unexpectedly.
- Verify images scale without distortion.
- Check touch targets and spacing around controls.
- Test headings, focus order, and form labels.
- Inspect contrast and readability in light and dark surroundings.

5.5 Capstone 1 — Personal Web Portfolio

Your first capstone proves that you can structure and present information professionally. It should not be a random template with your name inserted. The portfolio should communicate who you are, what you are learning, what you can build, and how someone can inspect your work.

- Hero/introduction with a concise professional message.
- About section focused on your transition or development goals.
- Technical skills grouped honestly by current proficiency.
- Projects section with screenshots, descriptions, technologies, and links.
- Contact or professional links.
- Responsive layout with usable navigation.
- README describing setup, design decisions, and learning reflections.

Capstone 1 completion checklist

1. Validate semantic HTML and fix obvious structure problems.
2. Apply consistent CSS or Bootstrap styling.
3. Test from narrow mobile widths through desktop widths.
4. Confirm links, images, and navigation all work.
5. Update README with project summary and technologies.
6. Commit a release-ready version and push to GitHub.
7. Deploy the static site using the instructor-approved platform.

Expected result

A public, responsive portfolio and repository suitable for future project additions.

Common beginner mistakes

Using Bootstrap classes without understanding them — Inspect the generated layout and learn the grid and spacing system.

Hiding content on mobile to “make it fit” — Reflow and prioritize content instead of deleting essential information.

Publishing placeholder text or broken links — Run a pre-deployment checklist and test the deployed URL, not only localhost.

Overclaiming skills — A credible junior portfolio is specific about what you actually built and learned.

Check your understanding

1. What does mobile-first mean in CSS?
2. Why should breakpoints be based on content behavior?
3. What is the role of Bootstrap's container/row/column system?
4. What should a portfolio project description communicate?
5. What tests should you perform after deployment?

SESSION 06

JavaScript Fundamentals

Stage 3 — JavaScript Essentials

Learning objectives

- ✓ Explain how JavaScript values, variables, operators, and expressions work.
- ✓ Use conditionals and loops to control program flow.
- ✓ Write functions with parameters, return values, and local scope.
- ✓ Use console output and debugging techniques to investigate logic.
- ✓ Build a small program that validates input and computes a result.

6.1 JavaScript changes a page from document to program

HTML and CSS can describe and present a page, but JavaScript can make decisions, react to events, transform data, request information, and update the interface. The same language will later run in Node.js on the server, which is why JavaScript provides a practical bridge from front end to back end in this bootcamp.

6.2 Values and data types

Type	Example	Use
string	"MSTCONNECT"	Text
number	42, 3.14	Numeric calculations
boolean	true / false	Conditions and flags
undefined	let result;	Value not assigned
null	let selected = null;	Intentional absence of value
object	{ name: "Ana" }	Structured data
array	[10, 20, 30]	Ordered collection

6.3 Variables: let and const

A variable is a named reference to a value. Use `const` by default when the binding should not be reassigned; use `let` when reassignment is part of the intended logic. `const` does not make objects deeply immutable—it only prevents assigning a different value to that variable name.

```
const bootcampName = "MSTCONNECT PH";  
let completedSessions = 5;  
completedSessions = completedSessions + 1;  
  
const learner = { name: "Alex" };  
learner.name = "Alex D."; // allowed: object contents changed
```

6.4 Operators and comparisons

Expressions combine values and operators to produce new values. Arithmetic operators calculate, comparison operators produce booleans, and logical operators combine conditions. Prefer strict equality `===` because it compares type as well as value and avoids surprising automatic conversions.

```
const score = 82;  
const attendance = 0.9;  
const passed = score >= 75 && attendance >= 0.8;  
  
console.log(passed); // true  
console.log(5 === "5"); // false
```

6.5 Conditionals

Conditionals choose which code runs. Good conditionals express the business rule clearly. Avoid deeply nested logic when early returns or helper functions can make the decision easier to read.

```
function getResult(average) {  
  if (average >= 90) return "Excellent";  
  if (average >= 75) return "Passed";  
  return "Needs improvement";  
}
```

6.6 Loops

Loops repeat work. Use `for...of` when you want to process each item in an iterable; traditional `for` loops give index control; `while` is useful when repetition depends on a condition. Always ask what changes each iteration and what makes the loop stop.

```
const scores = [80, 74, 91];  
let total = 0;  
  
for (const score of scores) {  
  total += score;  
}  
  
const average = total / scores.length;
```

6.7 Functions, parameters, and return values

A function packages behavior into a reusable unit. Parameters are local names for incoming values. `return` sends a result back to the caller and stops the current function. Prefer functions that do one clear job; small functions are easier to test and reuse.

```
function calculateAverage(scores) {  
  let total = 0;  
  for (const score of scores) total += score;  
  return total / scores.length;  
}  
  
const result = calculateAverage([80, 74, 91]);  
console.log(result);
```

6.8 Scope and debugging

Scope controls where a variable can be accessed. Variables declared inside a function or block are not automatically available everywhere. When debugging, inspect actual values at the moment the program misbehaves. Use `console.log` for simple tracing and browser breakpoints for step-by-step inspection.

Guided lab: learner score evaluator

1. Create a JavaScript file containing a learner name and three numeric scores.
2. Write `calculateAverage(scores)` and `getResult(average)` functions.
3. Validate that every score is between 0 and 100.
4. Print the learner name, average, and result.
5. Test boundary cases such as 74.99, 75, and 100.
6. Commit the exercise with a meaningful message.

Expected result

A small program with functions, validation, conditions, and test cases.

Common beginner mistakes

Confusing assignment = with comparison — Use = to assign and === to compare.

Returning nothing from a function that should produce a result — Trace where the return value is consumed.

Creating accidental strings from form values — Browser input values are text; convert to numbers when needed.

Fixing code by random edits — Change one hypothesis at a time and retest known cases.

Check your understanding

1. When should you choose const over let?
2. What is the difference between === and ==?
3. What does return do inside a function?
4. How do you know a loop will terminate?
5. Why are boundary test cases important?

SESSION 07

Arrays, Objects, and Modern JavaScript

Stage 3 — JavaScript Essentials

Learning objectives

- ✓ Model collections with arrays and structured records with objects.
- ✓ Use map, filter, find, and reduce-style patterns to transform data.
- ✓ Use destructuring, template literals, spread syntax, and arrow functions.
- ✓ Recognize mutation versus creating new arrays/objects.
- ✓ Organize code into reusable data-processing functions.

7.1 Arrays and objects model application data

A real application rarely works with one isolated value. It manages lists of users, products, tasks, orders, and messages. Arrays preserve ordered collections; objects group named fields describing one entity. Arrays of objects are a common shape across front-end state, JSON APIs, and MongoDB results.

```
const products = [  
  { id: 1, name: "Mechanical Keyboard", price: 2500, inStock: true },  
  { id: 2, name: "USB-C Hub", price: 1200, inStock: false },  
  { id: 3, name: "Webcam", price: 1800, inStock: true }  
];
```

7.2 Core array methods

Method	Question it answers
map	What should each item become?
filter	Which items should remain?
find	What is the first item that matches?
some	Does at least one item match?
every	Do all items match?
forEach	What side effect should run for each item?
reduce	How do many items combine into one accumulated result?


```
const names = products.map(product => product.name);
const available = products.filter(product => product.inStock);
const selected = products.find(product => product.id === 2);
const total = products.reduce((sum, product) => sum + product.price, 0);
```

7.3 Object access and destructuring

Objects can be accessed with dot notation when the property name is known, or bracket notation when the key is dynamic. Destructuring extracts properties into local variables and makes function parameters more expressive.

```
const product = { name: "Webcam", price: 1800, category: "Accessories" };
const { name, price } = product;

function formatProduct({ name, price }) {
  return `${name} - PHP ${price}`;
}
```

7.4 Spread syntax and immutability habits

Spread syntax copies enumerable values into a new array or object. This is useful when you want an updated value without modifying the original structure, a habit that becomes especially important in React state updates.

```
const updatedProduct = { ...product, price: 1750 };
const moreProducts = [...products, { id: 4, name: "Mouse", price: 900 }];
```

7.5 Arrow functions and callbacks

A callback is a function passed into another function so it can be invoked later or for each item. Array methods use callbacks extensively. Arrow functions provide compact syntax and capture this differently from traditional functions, but for beginner data transformations they are mainly a concise way to express callbacks.

7.6 Template literals

```
const learner = "Mika";
const completed = 7;
const message = `${learner} has completed ${completed} sessions.`;
```

Template literals use backticks and `${...}` expressions to combine text with values. They reduce the awkward string concatenation found in older code and are widely used in UI rendering and logging.

Guided lab: product data explorer

1. Create an array of at least five product objects with id, name, price, category, and inStock.
2. Create a function that returns all product names.
3. Filter products currently in stock.
4. Find one product by id.
5. Calculate the total or average price.
6. Return products belonging to a selected category.
7. Refactor repeated property access with destructuring and template literals.

Expected result

Reusable data-processing functions that can later power a DOM or React interface.

Common beginner mistakes

Using map when you only want to filter — Choose the method based on the result shape you need.

Forgetting that find may return undefined — Handle the case where no item matches.

Mutating shared objects unexpectedly — Create copies when later code depends on the original value.

Writing one huge callback — Extract complex logic into a named helper function.

Check your understanding

1. When would you use map instead of forEach?
2. What does filter return?
3. How does find behave when no item matches?
4. Why is spread syntax useful before React?
5. What is the advantage of arrays of objects for application data?

SESSION 08

DOM Manipulation, Events, JSON, and Browser Storage

Stage 3 — JavaScript Essentials

Learning objectives

- ✓ Explain the DOM as the browser's object representation of HTML.
- ✓ Select, create, modify, and remove elements safely.
- ✓ Handle click, input, and submit events.
- ✓ Serialize data with JSON and persist simple state in localStorage.
- ✓ Understand the role of fetch and asynchronous requests at a beginner level.

8.1 The DOM connects JavaScript to the page

After the browser parses HTML, it exposes the document as objects that JavaScript can inspect and change. This model is the Document Object Model (DOM). When JavaScript updates text, classes, attributes, or nodes, the browser recalculates the affected presentation and the user sees the change.

```
const heading = document.querySelector("h1");
heading.textContent = "Welcome back!";

const button = document.querySelector("#add-task");
button.addEventListener("click", () => {
  console.log("Button clicked");
});
```

8.2 Selecting and changing elements

- document.querySelector(selector): first matching element.
- document.querySelectorAll(selector): static collection of matches.
- textContent: set or read plain text safely.
- classList.add/remove/toggle: change classes without rewriting the whole class attribute.
- setAttribute/getAttribute: manage attributes when property access is not appropriate.
- createElement/append/remove: build and modify DOM structure.

8.3 Events and event objects

Events describe something that happened: a click, key press, input change, form submission, or network completion. `addEventListener` registers a callback. The event object contains context such as the target element. For forms, `preventDefault` stops the browser's automatic navigation so JavaScript can validate or process the submission first.

```
const form = document.querySelector("#task-form");

form.addEventListener("submit", event => {
  event.preventDefault();
  const input = document.querySelector("#task-title");
  const title = input.value.trim();
  if (!title) return;
  // add task
});
```

8.4 JSON

JSON is a text format for representing structured data. It resembles JavaScript object syntax but follows stricter rules. Web APIs frequently send and receive JSON because it is language-independent and easy to parse.

```
const tasks = [{ id: 1, title: "Practice DOM", done: false }];

const jsonText = JSON.stringify(tasks);
const restoredTasks = JSON.parse(jsonText);
```

8.5 localStorage

`localStorage` is browser storage scoped to a site origin. It stores strings and persists after a page reload, which makes it useful for small learning projects such as theme preferences or task lists. It is not a database replacement and should not store secrets.

```
localStorage.setItem("tasks", JSON.stringify(tasks));

const saved = localStorage.getItem("tasks");
const parsed = saved ? JSON.parse(saved) : [];
```

8.6 Introduction to fetch and asynchronous work

Network requests take time, so JavaScript does not block the entire browser waiting for them. `fetch` returns a Promise representing a future result. `async/await` gives asynchronous code a readable top-to-bottom shape. You will use this heavily when React connects to your API.

```
async function loadUsers() {  
  const response = await fetch("https://example.com/api/users");  
  if (!response.ok) throw new Error(`HTTP ${response.status}`);  
  const data = await response.json();  
  return data;  
}
```

Guided lab: persistent task manager

1. Create a form with one task input and an Add button.
2. On submit, validate the title and add a task object to an array.
3. Render the current array into the DOM.
4. Add a Complete action and a Delete action.
5. Save the task array to localStorage after every change.
6. Restore tasks from localStorage when the page loads.
7. Use DevTools Application/Storage panel to inspect the saved JSON.

Expected result

An interactive browser application whose data survives a page refresh.

Common beginner mistakes

Recreating event listeners repeatedly during render — Separate setup from rendering or use event delegation where appropriate.

Storing objects directly in localStorage — Serialize with `JSON.stringify` and parse on retrieval.

Using innerHTML for untrusted user input — Prefer `textContent` or safe element creation to avoid injection risks.

Assuming fetch rejects every HTTP error — Check `response.ok` or `status`; `fetch` can resolve for 404/500 responses.

Check your understanding

1. What is the DOM?
2. Why use `preventDefault` on a JavaScript-managed form?
3. What is the difference between `JSON.stringify` and `JSON.parse`?
4. Why is `localStorage` not a secure place for secrets?
5. Why do network requests require asynchronous code?

SESSION 09

Node.js, npm, Express, and Server Fundamentals

Stage 4 — Back-End Engineering

Learning objectives

- ✓ Explain how Node.js runs JavaScript outside the browser.
- ✓ Use npm and package.json to manage dependencies and scripts.
- ✓ Create an Express server with routes and JSON responses.
- ✓ Explain middleware and the request/response lifecycle.
- ✓ Use environment variables for configuration.
- ✓ Test local server routes in the browser and Postman.

9.1 What Node.js changes

Browser JavaScript has APIs for documents, windows, storage, and user events. Node.js runs JavaScript in a server/command-line environment and provides different APIs for files, processes, networking, streams, and operating-system interaction. This allows one language to be used on both sides of the web application, while the runtime environments remain different.

9.2 npm and package.json

npm is a package manager and script runner commonly used with Node.js. package.json describes the project and declares dependencies. package-lock.json records exact resolved package versions so installations are more reproducible. node_modules contains installed packages and is normally excluded from Git because it can be recreated with npm install.

```
mkdir ecommerce-api
cd ecommerce-api
npm init -y
npm install express

# package.json can define scripts such as:
# "dev": "node server.js"
```

9.3 Creating an Express server

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.get("/", (req, res) => {
  res.json({ message: "API is running" });
});

app.listen(PORT, () => {
  console.log(`Server listening on http://localhost:${PORT}`);
});
```

An Express application is a chain of request handlers. `app.get` defines a handler for GET requests at a path. `req` contains request data, `res` provides methods for constructing the response, and `next` is used by middleware to pass control forward.

9.4 Routes and route parameters

```
app.get("/api/products/:id", (req, res) => {
  const id = req.params.id;
  res.json({ requestedId: id });
});
```

A route combines an HTTP method and path. Route parameters identify part of the path, while query parameters are typically used for optional filtering, sorting, pagination, or search. For example, `/api/products/123` identifies a product, while `/api/products?category=keyboard` modifies a collection query.

9.5 Middleware

Middleware is code that runs during the request pipeline. It can parse JSON, log requests, authenticate users, validate input, attach data to `req`, handle errors, or perform cross-cutting concerns. Ordering matters because middleware runs in the order it is registered.

```
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});
```

9.6 Environment variables and configuration

Configuration values differ between machines and environments. Secrets should not be hard-coded into source files. Environment variables allow the process to read values such as database connection strings, API keys, and ports at runtime. A local `.env` file may be used during development but should be excluded from version control.

```
.env
PORT=3000
MONGODB_URI=mongodb+srv://...

.gitignore
node_modules/
.env
```

9.7 Error handling and 404s

A server should return predictable responses when something goes wrong. Unknown routes should return 404, invalid input should return a 4xx status, and unexpected server failures should be logged and handled without exposing secrets or stack traces to production users.

Guided lab: first Express API

1. Create a new Node project and install Express.
2. Add `express.json()` and a request-logging middleware.
3. Create `GET /` and `GET /api/products` routes.
4. Create `GET /api/products/:id` using an in-memory product array.
5. Return 404 when a product id does not exist.
6. Test every route in Postman and inspect response status/body.
7. Commit the API starter project to GitHub.

Expected result

A local Express server with JSON routes, middleware, and basic error responses.

Common beginner mistakes

Running node from the wrong directory — Use `pwd/ls` and verify `package.json` is in the current folder.

Forgetting `express.json()` — JSON request bodies will be undefined unless parsing middleware runs.

Sending more than one response — A request should finish once; return after sending in conditional branches.

Committing `node_modules` or `.env` — Use `.gitignore` before the first push.

Check your understanding

1. What does Node.js provide that browser JavaScript does not?
2. What is the purpose of `package.json`?
3. What makes a route different from middleware?
4. Why does middleware order matter?
5. Why should secrets come from environment variables?

SESSION 10

MongoDB, Mongoose, and Database Operations

Stage 4 — Back-End Engineering

Learning objectives

- ✓ Explain documents, collections, and how MongoDB differs from relational tables conceptually.
- ✓ Connect a Node application to MongoDB using Mongoose.
- ✓ Define schemas and models with validation.
- ✓ Perform create, read, update, and delete operations.
- ✓ Handle missing records and validation errors predictably.

10.1 Why databases exist

Variables disappear when a process ends. Databases provide durable, queryable storage that multiple requests and users can share. A database also provides indexing, constraints/validation, concurrency behavior, and operational tools that a plain JSON file does not.

10.2 MongoDB document model

MongoDB stores BSON documents in collections. Documents resemble JavaScript objects and can contain nested objects and arrays. This makes it natural for JavaScript developers, but flexible structure is not an excuse for uncontrolled data. Good applications still define expected fields and validation rules.

```
{
  "_id": "...",
  "name": "Mechanical Keyboard",
  "price": 2500,
  "category": "Accessories",
  "stock": 12,
  "tags": ["keyboard", "usb"]
}
```

10.3 Mongoose schemas and models

Mongoose is an object modeling library for MongoDB. A schema describes fields, types, defaults, and validation. A model is the application interface used to create and query documents in a collection.

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  description: { type: String, default: "" },
  price: { type: Number, required: true, min: 0 },
  category: { type: String, required: true },
  stock: { type: Number, default: 0, min: 0 }
}, { timestamps: true });

module.exports = mongoose.model("Product", productSchema);
```

10.4 Connecting to MongoDB

```
async function connectDatabase() {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log("MongoDB connected");
  } catch (error) {
    console.error("Database connection failed", error.message);
    process.exit(1);
  }
}
```

The connection string belongs in an environment variable. In a hosted database service, access controls may include database users, network rules, and TLS. If a connection fails, inspect the exact error and verify credentials, URI formatting, network access, and database availability rather than randomly changing code.

10.5 CRUD operations

```
// Create
const product = await Product.create(req.body);

// Read many
const products = await Product.find();

// Read one
const productById = await Product.findById(req.params.id);

// Update
const updated = await Product.findByIdAndUpdate(
  req.params.id,
  req.body,
  { new: true, runValidators: true }
);

// Delete
const deleted = await Product.findByIdAndDelete(req.params.id);
```

10.6 Validation belongs at multiple layers

The front end should guide users, the API should reject invalid requests, and the data model should prevent invalid persistent state. Multiple layers are not redundant: each protects a different boundary. Never assume a browser form is the only way your API can be called.

Guided lab: persistent product model

1. Create a MongoDB database and obtain the connection URI.
2. Add MONGODB_URI to your local environment configuration.
3. Create a Product schema with required fields and numeric validation.
4. Connect the Express application to MongoDB.
5. Replace the in-memory GET route with Product.find().
6. Create routes that insert, update, and delete products.
7. Test invalid price/stock values and missing ids.

Expected result

An Express API whose product data persists in MongoDB.

Common beginner mistakes

Hard-coding the connection URI — Use environment variables and keep .env out of Git.

Assuming an id always exists — Handle null results and invalid ObjectId formats.

Updating without validators — Use runValidators: true where appropriate and validate request bodies.

Treating a flexible document model as “no schema needed” — Define application-level data contracts so your API remains predictable.

Check your understanding

1. Why is database persistence different from an in-memory array?
2. What is the difference between a Mongoose schema and model?
3. What does CRUD stand for?
4. Why should update operations run validation?
5. Why should the API validate even if the front end already validates?

SESSION 11

REST API Design, Postman Testing, and Capstone 2

Stage 4 — Back-End Engineering

Learning objectives

- ✓ Design resource-oriented REST endpoints and choose suitable HTTP methods.
- ✓ Return meaningful status codes and consistent JSON responses.
- ✓ Separate routes, controllers, models, and middleware for maintainability.
- ✓ Test positive and negative cases with Postman.
- ✓ Document an API so another developer can use it.
- ✓ Complete Capstone 2: Functional E-Commerce REST API.

11.1 REST as a resource-oriented style

REST is an architectural style rather than a single library. In practical API work, a RESTful design represents application concepts as resources and uses standard HTTP semantics. URLs should generally identify nouns/resources while HTTP methods express actions.

Operation	Method	Endpoint	Typical success status
List products	GET	/api/products	200 OK
Read product	GET	/api/products/:id	200 OK
Create product	POST	/api/products	201 Created
Update product	PATCH	/api/products/:id	200 OK
Delete product	DELETE	/api/products/:id	204 No Content or 200 OK

11.2 Status codes are part of the contract

Status	Meaning	Typical use
200	OK	Successful GET or update response.
201	Created	Resource successfully created.
204	No Content	Successful response with no body.
400	Bad Request	Malformed or invalid client data.
401	Unauthorized	Authentication required/invalid.
403	Forbidden	Authenticated but not allowed.
404	Not Found	Resource or route does not exist.
409	Conflict	Request conflicts with current state.
500	Internal Server Error	Unexpected server failure.

11.3 Request data: params, query, body, headers

Different parts of a request answer different questions. Route parameters identify the resource, query parameters modify a collection request, the body carries submitted data, and headers carry metadata such as content type or authorization.

```
GET /api/products?category=accessories&limit=10

POST /api/products
Content-Type: application/json

{
  "name": "USB-C Hub",
  "price": 1200,
  "category": "Accessories",
  "stock": 20
}
```

11.4 Separating responsibilities

A beginner server often starts in one file. As features grow, separate responsibilities so routes define endpoints, controllers coordinate request logic, models handle data structure/persistence, and middleware handles shared concerns. This makes code easier to test and prevents one file from becoming an unmaintainable wall of logic.

```
src/  
├── models/  
│   └── Product.js  
├── controllers/  
│   └── productController.js  
├── routes/  
│   └── productRoutes.js  
├── middleware/  
│   └── errorHandler.js  
└── app.js
```

11.5 Postman as a repeatable test client

Postman lets you construct requests independently of the front end. This is important because you can test the API before React exists. Save requests in a collection, use environment variables for base URLs, and test failure cases deliberately—not only the happy path.

- Create valid data and confirm 201 plus returned document.
- Send missing required fields and confirm a 4xx response.
- Request a nonexistent id and confirm 404.
- Update with an invalid price and confirm validation prevents it.
- Delete a record and confirm subsequent GET returns 404.
- Restart the server and confirm database data persists.

11.6 Authentication concepts

Authentication answers “Who are you?” Authorization answers “What are you allowed to do?” Production authentication requires careful password hashing, token/session handling, transport security, expiration, revocation, and access-control design. This bootcamp introduces the concepts so you can recognize where they belong; do not invent your own cryptography or store plain-text passwords.

11.7 Capstone 2 — E-Commerce REST API

- Product CRUD endpoints with MongoDB persistence.
- Validation for names, prices, stock, and required fields.
- Consistent success and error responses.
- Search or filtering through query parameters.
- Optional categories, users, or order structure depending on available time.
- Postman collection covering success and failure cases.
- README with setup, environment variables, endpoint documentation, and example requests.

Capstone 2 quality gate

1. Run every CRUD endpoint from a clean Postman collection.
2. Test at least one invalid input for every write endpoint.
3. Confirm status codes and response shapes are consistent.
4. Verify secrets are not present in Git history or README.
5. Document required environment variables without exposing their values.
6. Push a clean repository with setup steps that a classmate can follow.

Expected result

A functioning, documented, testable e-commerce API ready for React integration.

Common beginner mistakes

Using verbs in every URL such as /getProducts — Prefer resource nouns and let HTTP methods express intent.

Returning 200 for every situation — Use status codes to communicate the actual outcome.

Testing only valid requests — Real API quality becomes visible in error handling and edge cases.

Logging secrets or raw credentials — Keep logs useful without exposing sensitive data.

Check your understanding

1. What makes an endpoint resource-oriented?
2. What is the difference between 401 and 403?
3. When should a value be a route parameter versus query parameter?
4. Why should API tests include failure cases?
5. What information does another developer need in API documentation?

SESSION 12

React Fundamentals: Components, Props, and State

Stage 5 — Full-Stack with React

Learning objectives

- ✓ Explain why component-based UI helps manage larger interfaces.
- ✓ Write JSX and functional components.
- ✓ Pass data through props and manage changing values with state.
- ✓ Render lists with stable keys and handle user events.
- ✓ Use conditional rendering to represent different UI states.

12.1 Why React?

Plain DOM scripting is excellent for learning browser fundamentals, but larger interfaces become difficult when many UI regions depend on changing data. React provides a component model: you describe what the UI should look like for the current state, and React coordinates updates when state changes. This encourages composition, reuse, and predictable data flow.

12.2 JSX

JSX is syntax that lets JavaScript expressions describe UI structure. It resembles HTML but follows JavaScript rules: `className` instead of `class`, expressions inside curly braces, and components beginning with uppercase names. JSX is transformed by the build tool into JavaScript React calls.

```
function Welcome({ name }) {  
  return (  
    <section className="welcome">  
      <h1>Hello, {name}</h1>  
      <p>Ready to build?</p>  
    </section>  
  );  
}
```

12.3 Components and composition

A component should represent a meaningful UI responsibility: ProductCard, Navbar, SearchBar, ProjectList. Composition means assembling components to form larger screens. Avoid splitting every single line into a component; extract units that have a clear role, repeated pattern, or independent logic.

12.4 Props

Props are inputs passed from a parent component to a child. They make components reusable because the structure can remain the same while data changes. Props should be treated as read-only from the receiving component's perspective.

```
function ProductCard({ product }) {  
  return (  
    <article>  
      <h2>{product.name}</h2>  
      <p>PHP {product.price}</p>  
    </article>  
  );  
}  
  
<ProductCard product={products[0]} />
```

12.5 State with useState

State represents information that can change over time and should trigger a re-render. useState returns the current value and a setter function. Do not mutate state objects/arrays directly; create a new value so React can reason about the update.

```
const [query, setQuery] = useState("");  
  
<input  
  value={query}  
  onChange={event => setQuery(event.target.value)}  
>
```

12.6 Lists and keys

Rendering a list is usually a map from data to components. Each sibling needs a stable key that identifies the item across renders. Prefer a database id or other durable identifier; array indexes can cause subtle UI bugs when items are inserted, removed, or reordered.

```
{products.map(product => (  
  <ProductCard key={product.id} product={product} />  
))}
```

12.7 Conditional rendering

Interfaces must represent multiple states: no data, loading, error, permission denied, success, or filtered results. Conditional rendering expresses these states explicitly instead of assuming everything always succeeds.

```
if (products.length === 0) {  
  return <p>No products found.</p>;  
}
```

Guided lab: React product gallery

1. Create a React project with the instructor-approved Vite setup.
2. Create Header, SearchBar, ProductCard, and ProductList components.
3. Pass product data into cards through props.
4. Store search text in state and filter the visible product array.
5. Render the filtered list with stable keys.
6. Show an empty-state message when no products match.

Expected result

A component-based interactive gallery using props, state, events, and conditional rendering.

Common beginner mistakes

Changing props inside a child — Ask the parent to update state via callbacks instead.

Mutating arrays with push and then setting the same reference — Create a new array using spread, map, or filter.

Using an unstable key — Use a real item id whenever possible.

Putting all logic in App.jsx — Extract components and helper functions around clear responsibilities.

Check your understanding

1. What problem does a component model solve?
2. What is the difference between props and state?
3. Why should React state updates create new arrays/objects?
4. Why do list items need stable keys?
5. What UI states should a data-driven page represent?

SESSION 13

React Forms, Routing, Effects, and API Integration

Stage 5 — Full-Stack with React

Learning objectives

- ✓ Build controlled React forms with validation and submission handling.
- ✓ Use React Router to organize multiple views and dynamic routes.
- ✓ Use `useEffect` for synchronizing with external systems such as APIs.
- ✓ Load and submit data through HTTP requests.
- ✓ Represent loading, error, empty, and success states.
- ✓ Connect the React front end to the Express/MongoDB API.

13.1 Controlled forms

In a controlled form, React state is the source of truth for input values. The input displays state, and `onChange` updates state. This makes validation, conditional UI, reset behavior, and submission logic predictable.

```
const [form, setForm] = useState({ name: "", price: "" });

function handleChange(event) {
  const { name, value } = event.target;
  setForm(current => ({ ...current, [name]: value }));
}
```

13.2 Routing

Routing maps URLs to interface views. A single-page React application can update content without a full browser reload while still using meaningful URLs. Dynamic route parameters support detail pages such as `/products/:id`. Good routing improves navigation, deep linking, browser history, and project organization.

```
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/products" element={<ProductsPage />} />
  <Route path="/products/:id" element={<ProductDetailsPage />} />
</Routes>
```

13.3 useEffect: synchronization, not general-purpose logic

`useEffect` runs after render to synchronize a component with something outside React's pure rendering process: network requests, timers, subscriptions, or imperative browser APIs. Do not use effects for calculations that can be derived directly from props/state. Dependency arrays tell React when the effect must be re-synchronized.

```
useEffect(() => {
  async function loadProducts() {
    try {
      setLoading(true);
      const response = await fetch(`${API_URL}/api/products`);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      setProducts(await response.json());
    } catch (error) {
      setError(error.message);
    } finally {
      setLoading(false);
    }
  }

  loadProducts();
}, []);
```

13.4 Loading, error, empty, success

A network-backed UI is not just “data” or “no data.” Users need to know whether the application is still loading, encountered an error, received an empty result, or succeeded. Modeling these states explicitly makes interfaces resilient and easier to debug.

```
if (loading) return <p>Loading products...</p>;
if (error) return <p role="alert">Could not load: {error}</p>;
if (products.length === 0) return <p>No products available.</p>;
```

13.5 Sending data to the API

```
const response = await fetch(`${API_URL}/api/products`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    ...form,
    price: Number(form.price)
  })
});
```

The front end should validate for user experience, but the API remains the authority for acceptable data. After a successful write, update local state from the server response or reload the relevant data. Do not assume the server stored exactly what the user typed; server-side defaults or normalization may change the final document.

13.6 CORS and environment configuration

During development, the React dev server and Express API often run on different origins, so the browser's same-origin policy may block requests unless the server explicitly allows the front-end origin through CORS configuration. CORS is a browser security policy, not authentication. Front-end environment variables should contain non-secret configuration such as public API base URLs; anything shipped to the browser can be inspected by users.

13.7 Capstone 3 — React full-stack application

- At least three meaningful routes.
- Reusable component structure.
- API data loading with visible loading/error/empty states.
- Create or edit form connected to the API.
- Dynamic detail view or equivalent data-driven route.
- Responsive layout and accessible controls.
- README explaining front-end and back-end setup.
- Deployed front end and API where feasible.

Guided lab: connect React to Capstone 2

1. Configure the API base URL for development.
2. Create a Products page that loads data from GET /api/products.
3. Create a detail route that loads or selects one product by id.
4. Create a controlled form for adding products.
5. Submit the form to POST /api/products and update the UI.
6. Display user-friendly errors for failed requests.
7. Test the full request flow using browser Network tools and server logs.

Expected result

A React client that reads from and writes to the Express/MongoDB API.

Common beginner mistakes

Putting secrets in front-end environment variables — Anything bundled into browser code is public.

Creating an infinite effect loop — Understand dependencies and avoid effects that update the state they depend on without a stopping condition.

Ignoring response.ok — Handle HTTP errors before assuming response.json contains valid success data.

Blaming React when the API is broken — Test the API directly in Postman, then inspect the browser Network request.

Check your understanding

1. What makes a React input controlled?
2. When is useEffect appropriate?
3. Why should loading, error, and empty states be separate?
4. What does CORS control?
5. Why must server responses remain the source of truth after a write?

SESSION 14

Testing, Debugging, Documentation, and Deployment

Stage 6 — Capstone Projects

Learning objectives

- ✓ Use a systematic debugging workflow across front end, network, server, and database layers.
- ✓ Perform functional, responsive, accessibility, and regression checks.
- ✓ Refactor obvious duplication without changing behavior.
- ✓ Write setup-focused README documentation.
- ✓ Prepare projects for deployment and live demonstration.
- ✓ Audit repositories for secrets, broken links, and unfinished work.

14.1 Debugging is a method, not a mood

A full-stack bug can live in the browser, React state, request payload, CORS settings, Express route, validation, database query, environment configuration, or deployment. Random editing is slow because several layers may appear to fail when only one is wrong. Systematic debugging narrows the problem boundary.

1. Reproduce the problem reliably.
2. State the expected behavior and the actual behavior.
3. Locate the failing layer: UI, request, server, database, or environment.
4. Inspect evidence: console, Network panel, server logs, Postman, database data.
5. Create one hypothesis.
6. Change one variable and retest.
7. Add a regression test/check so the same problem is less likely to return.

Debugging question

Can the API endpoint work correctly in Postman?
If yes, investigate the React request. If no,
investigate the server/database before changing
the front end.

14.2 Testing categories for bootcamp projects

Category	Examples
Functional	Create, update, filter, delete, navigation, form validation.
Negative/error	Missing fields, bad id, offline API, database failure, empty result.
Responsive	Narrow/mobile widths, tablet, desktop, long content.
Accessibility	Keyboard access, labels, focus, headings, contrast.
Regression	Previously working features still work after changes.
Deployment	Public URLs, environment variables, CORS, refresh on routes.

14.3 Refactoring safely

Refactoring changes code structure without intentionally changing behavior. Good reasons include duplicated logic, unclear names, giant functions, or repeated markup. Refactor after you have a working baseline and test after each small change. Do not perform a massive rewrite immediately before demo day.

14.4 README as operational documentation

- Project name and problem statement.
- Screenshots or demo link.
- Technology stack.
- Features and known limitations.
- Prerequisites and installation steps.
- Environment variables by name only—never secret values.
- Commands to run front end and back end.
- API summary or link to endpoint documentation.
- Author, learning reflection, and planned improvements.

14.5 Deployment concepts

Deployment moves the project from your local development environment to infrastructure that users can reach. Front-end deployment serves built static assets; API deployment runs a server process; database hosting provides persistent storage. Environment variables, build commands, allowed origins, ports, and URL configuration must be correct for the target platform.

- Use production environment variables instead of local `.env` values.
- Update the React API base URL to the deployed API endpoint.
- Configure CORS to allow only the intended front-end origin where practical.
- Verify SPA route refresh behavior for React Router.
- Check server logs after deployment instead of assuming a failed request is a front-end issue.
- Test the deployed system from a separate browser/device when possible.

14.6 Security hygiene before publishing

- Search the repository for passwords, tokens, connection strings, and private files.
- Confirm `.env` and `node_modules` are ignored.
- Rotate any credential that was accidentally committed; deleting the line later is not enough.
- Avoid exposing detailed internal errors to end users.
- Validate inputs server-side and do not trust browser-side checks.
- Keep dependencies and runtime versions supported and documented.

Capstone audit sprint

1. Run a feature checklist for all three capstones.
2. Fix console errors and failed network requests.
3. Test responsive behavior and keyboard navigation.
4. Review Git history and remove unnecessary generated files.
5. Complete each README and verify setup steps from a clean folder if possible.
6. Prepare sample/demo data that makes the project easy to present.
7. Deploy and test every public URL.
8. Prepare backup screenshots or a short local demo plan in case connectivity fails.

Expected result

Three portfolio-ready repositories with clear documentation and a reliable demo path.

Common beginner mistakes

Making large new features on the final night — Prioritize reliability, clarity, and documentation over last-minute scope.

Testing only on localhost — Deployment introduces new URLs, CORS, environment, and routing behavior.

Deleting secrets without rotating them — Assume committed secrets may be exposed and replace them.

Writing README steps from memory — Follow the instructions yourself or have a classmate test them.

Check your understanding

1. What evidence should you inspect for a full-stack failure?
2. What makes regression testing different from initial testing?
3. Why is refactoring risky immediately before a deadline?
4. What belongs in deployment environment variables?
5. What should happen if a secret was committed to Git?

SESSION 15

Demo Day, GitHub Portfolio, and Career Launch

Stage 7 — Commencement & Career Launch

Learning objectives

- ✓ Present a project in a concise problem-solution-technology-demo format.
- ✓ Explain technical decisions and debugging experiences in plain language.
- ✓ Organize GitHub repositories for recruiter/client review.
- ✓ Translate bootcamp projects into résumé bullet points and interview stories.
- ✓ Build a realistic post-bootcamp practice and job-search plan.

15.1 What a strong project presentation proves

A demo is not only a tour of screens. It proves that you understand the problem, architecture, implementation, trade-offs, and current limitations. Employers expect junior developers to be able to explain what they personally built and how they investigated problems.

1. State the problem and intended user in one or two sentences.
2. Summarize the stack and why each layer exists.
3. Demonstrate the main user flow, not every minor feature.
4. Show one technical detail you understand well.
5. Describe a bug or challenge and how you debugged it.
6. Acknowledge limitations and name the next improvement.

15.2 GitHub as evidence

A GitHub profile is useful when it makes your work easy to evaluate. Pin your strongest repositories, write clear descriptions, keep READMEs complete, and avoid dozens of abandoned tutorial repositories appearing above your capstones. The goal is not artificial perfection; it is legibility and ownership.

- Use a recognizable profile name and professional bio.
- Pin the portfolio, API, and React full-stack project.
- Include live demo links where stable.
- Keep repository names and descriptions clear.

- Make recent commits meaningful rather than creating noise for activity graphs.
- Show progressive improvement through documentation and code organization.

15.3 Turning projects into résumé bullets

A useful résumé bullet combines what you built, the technologies used, and the outcome or capability. Avoid vague statements such as “Learned React.” Prefer evidence-focused language that can be defended in an interview.

Built a responsive portfolio using semantic HTML, CSS, Bootstrap, and JavaScript; deployed the site and documented setup in GitHub.

Developed a Node.js/Express REST API with MongoDB/Mongoose CRUD operations, validation, structured error responses, and Postman test collections.

Integrated a React front end with a REST API using routing, controlled forms, reusable components, loading/error states, and deployed environment configuration.

15.4 Technical interview mindset

Junior interviews usually evaluate fundamentals, reasoning, communication, and ability to learn. You do not need to answer every question instantly. Clarify assumptions, explain your thought process at a useful level, test examples, and admit uncertainty rather than inventing an answer.

- Explain HTML/CSS/JS roles clearly.
- Describe the client-server-database request flow.
- Explain Git staging, commits, branches, and remotes.
- Compare arrays/objects and common JavaScript methods.
- Explain REST methods, status codes, and validation.
- Explain props/state/effects and how React loads API data.
- Walk through one real debugging story from your capstone.

15.5 Career paths after the bootcamp

Full-stack training can support several entry paths: junior front-end developer, junior web developer, full-stack trainee, QA/automation-adjacent roles, implementation/support engineering, internal business systems, freelance websites, or internships. Your first role does not need to use every technology in the stack. Focus on roles where your portfolio provides credible evidence and where you can continue learning with mentorship.

15.6 A 90-day continuation plan

Period	Primary goal	Suggested work
Days 1-30	Strengthen fundamentals	Rebuild small HTML/CSS/JS exercises without notes; fix capstone issues.
Days 31-60	Increase project depth	Add authentication, testing, pagination, or one carefully scoped feature.
Days 61-90	Job readiness	Practice interviews, refine résumé/GitHub, apply consistently, contribute or collaborate.

Career principle

Do not wait until you “know everything.” Apply when you can explain fundamentals, demonstrate projects, debug common problems, and show a credible learning process.

Final demo checklist

1. Open all live URLs and repositories before presentation time.
2. Prepare a 3-5 minute story for your strongest project.
3. Demonstrate one complete user flow.
4. Explain one bug you solved with evidence.
5. Show README and Git history briefly.
6. State one improvement you would build next.
7. Update your GitHub profile and résumé after receiving feedback.

Expected result

A portfolio package that demonstrates working code, documentation, communication, and continued learning.

Common beginner mistakes

Presenting code you cannot explain — Choose fewer features and know them deeply.

Apologizing for being a beginner — Describe what you built, what you learned, and what you are improving.

Listing every technology ever encountered — Prioritize skills supported by project evidence.

Stopping practice after graduation — Schedule continued projects and applications while fundamentals are fresh.

Check your understanding

1. How would you explain your full-stack architecture to a nontechnical interviewer?
2. Which three repositories should you pin first, and why?
3. What is your strongest debugging story from the bootcamp?
4. Which role types best match your current evidence?
5. What will you build or improve in the first 30 days after graduation?

Appendix A — Windows and macOS Command Quick Reference

Task	Windows PowerShell	macOS Terminal
Current directory	pwd	pwd
List contents	ls	ls
Change directory	cd path	cd path
Go up	cd ..	cd ..
Home	cd ~	cd ~
Create folder	mkdir name	mkdir name
Remove empty folder	rmdir name	rmdir name
Clear terminal	cls	clear
Open VS Code here	code .	code .
Git version	git --version	git --version
Node version	node --version	node --version
npm version	npm --version	npm --version

macOS note

If code . is not recognized, open VS Code → Command Palette → “Shell Command: Install ‘code’ command in PATH”, then reopen Terminal.

Appendix B — Git Cheat Sheet

Goal	Command
Start repository	git init
Inspect status	git status
View unstaged diff	git diff
Stage one file	git add filename
Stage current project changes	git add .
Commit	git commit -m "message"
View history	git log --oneline
Create/switch branch	git switch -c branch-name
Switch branch	git switch main
Show branches	git branch
Show remotes	git remote -v
Add remote	git remote add origin URL
Push main first time	git push -u origin main
Push later commits	git push
Get remote changes	git pull

Appendix C — HTTP and REST Quick Reference

Concept	Quick reference
GET	Retrieve resource(s); should not intentionally modify server state.
POST	Create a resource or trigger an operation.
PUT	Replace/update a full resource representation.
PATCH	Partially update a resource.
DELETE	Remove a resource.
200	Request succeeded.
201	Resource created.
204	Success with no response body.
400	Invalid request data.
401	Authentication required or invalid.
403	Authenticated but not permitted.
404	Resource not found.
409	Conflict with current state.
500	Unexpected server failure.

Appendix D — HTML/CSS Starter Reference

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page Title</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
  <script src="js/script.js"></script>
</body>
</html>

*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; line-height: 1.5; }
.container { width: min(100% - 2rem, 1100px); margin-inline: auto; }
img { max-width: 100%; height: auto; }
```

Appendix E — JavaScript Starter Reference

```
const items = [1, 2, 3];
const doubled = items.map(item => item * 2);
const even = items.filter(item => item % 2 === 0);
const found = items.find(item => item === 2);

function greet(name) {
  return `Hello, ${name}`;
}

const button = document.querySelector("button");
button.addEventListener("click", () => console.log("clicked"));
```

Appendix F — Node/Express/Mongoose Starter Structure

```
project/
├── src/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   └── app.js
├── .env
├── .gitignore
├── package.json
└── README.md
```

Appendix G — React Mental Model

- Props: read-only inputs from a parent.
- State: changing component data that triggers rendering.
- Events: user actions that request state changes.
- Derived data: calculate from props/state instead of storing duplicate state when possible.
- Effects: synchronize with external systems such as APIs or timers.
- Router: map URLs to views.
- API layer: centralize repeated request behavior as applications grow.

Appendix H — Capstone Quality Checklist

Area	Questions to ask
Functionality	Do the main user flows work from start to finish?
Errors	Are validation and failure states understandable?
Responsive UI	Does the interface reflow without accidental horizontal scrolling?
Accessibility	Can users navigate controls with a keyboard? Are labels/headings meaningful?
Git	Are commits meaningful and repositories free of generated clutter/secrets?
README	Can another learner run the project from the instructions?
API	Are methods, status codes, validation, and error shapes consistent?
Database	Does data persist and do invalid records get rejected?
React	Are components reusable and data/loading/error states explicit?
Deployment	Do public URLs and environment configurations work outside your machine?

Appendix I — Glossary for New Developers

Term	Meaning
API	A defined interface that lets software systems communicate.
Branch	A named line of Git development.
Browser	Client software that retrieves and runs web content.
Build	A process that transforms source files into deployable output.
Client	Software/device initiating a request, commonly the browser.
Commit	A recorded Git snapshot with metadata.
Component	A reusable unit of UI and behavior in React.
CORS	Browser policy controlling cross-origin access to server responses.
CRUD	Create, Read, Update, Delete.
Database	Persistent system for storing and querying application data.
Dependency	External package/library a project relies on.
Deployment	Publishing software into a target environment.
DNS	System that maps domain names to network addresses.
DOM	Browser object representation of the HTML document.
Endpoint	A method + path exposed by an API.
Environment variable	Runtime configuration value outside source code.
HTTP	Protocol used for web requests and responses.

Term	Meaning
JSON	Text format for structured data exchange.
Middleware	Code that processes requests in the server pipeline.
Model	Application abstraction used to interact with persistent data.
npm	Package manager and script runner commonly used with Node.js.
Props	Inputs passed into a React component.
Repository	Git-managed project history.
Request	Message sent from client to server.
Response	Message returned from server to client.
Route	Mapping between request method/path and application logic.
Server	Software that listens for and responds to network requests.
State	Data that can change and influence rendered UI.
Validation	Checking whether data satisfies required rules.
Version control	System for recording and collaborating on file history.

Appendix J — Independent Practice Ladder

When you are unsure what to practice after class, move through this ladder. Repeat earlier levels whenever later work feels confusing.

1. Rebuild one static page from a screenshot using semantic HTML.
2. Style the page responsively without copying the original CSS.
3. Add JavaScript interactions such as filtering, validation, or a task list.
4. Store simple state in localStorage and restore it on reload.
5. Build an Express API around one resource.
6. Persist the resource in MongoDB with Mongoose validation.
7. Test the API systematically in Postman.
8. Build a React client that lists, creates, updates, and deletes the resource.
9. Deploy the front end and API, then debug environment/CORS issues.
10. Explain the complete request flow to another person without reading notes.

Further References and Readings

The resources below are intended to complement the MSTCONNECT PH Full-Stack Web Development Participant Manual. Participants are **not expected to read every resource from beginning to end during the bootcamp**. Instead, use them as references when a lesson needs reinforcement, when completing exercises and capstones, and when continuing your learning after graduation.

A useful learning sequence is:

Manual → Instructor Demo → Hands-On Practice → Official Documentation → Independent Practice

Official documentation should generally be your first reference for technical behavior because frameworks and tools evolve.

A. Stage 1 — Developer Foundations

Sessions 1–2: How the Web Works, VS Code, Git, and GitHub

1. MDN — Learn Web Development

Recommended for: Beginners, career shifters, and refreshers

MDN provides one of the strongest structured introductions to web development. Its curriculum includes environment setup, command-line basics, browsers, HTML, CSS, JavaScript, accessibility, responsive design, deployment, workflows, and career-related skills.

Recommended reading:

- Getting Started modules
- Environment Setup
- Your First Website
- Web Standards
- How the Web Works
- Soft Skills
- Core Learning Modules

MDN Learn Web Development:

https://developer.mozilla.org/en-US/docs/Learn_web_development

2. MDN — Client-Server Overview

Use this after Session 1 when the client/server/request/response model still feels abstract.

Focus on:

- Web clients
- Web servers
- HTTP requests
- HTTP responses
- Static vs. dynamic websites
- Server-side frameworks

MDN Client-Server Overview:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview

3. MDN — HTTP

This becomes increasingly valuable during the JavaScript, API, Express, Postman, and React stages.

Recommended topics:

- HTTP overview
- Request methods
- Response status codes
- Headers
- HTTP messages
- Authentication
- Cookies
- Caching

MDN HTTP Documentation:

<https://developer.mozilla.org/en-US/docs/Web/HTTP>

4. Visual Studio Code Documentation

Use this when you need help with the editor rather than application code.

Recommended sections:

- Getting Started
- User Interface
- Integrated Terminal
- Extensions
- Settings
- Source Control
- Debugging
- Keyboard shortcuts

Visual Studio Code Documentation:

<https://code.visualstudio.com/docs>

5. Pro Git — Scott Chacon and Ben Straub

Recommended for: Everyone who wants a deeper understanding of Git

The complete *Pro Git* book is available through the official Git website. It covers version-control concepts, repositories, commits, branches, remotes, merging, collaboration, and advanced Git topics.

Start with:

- Chapter 1 — Getting Started
- Chapter 2 — Git Basics
- Chapter 3 — Git Branching

Return to the later chapters after the bootcamp.

Read Pro Git Online:

<https://git-scm.com/book/en/v2>

A Tagalog edition is also available.

Pro Git — Tagalog Edition:

<https://git-scm.com/book/tl/v2.html>

6. GitHub — Getting Started Documentation

Recommended topics:

- Creating a GitHub account
- Creating repositories
- Commits
- Branches
- Connecting local repositories
- GitHub Flow
- Pull requests
- Profile management

GitHub Getting Started Documentation:

<https://docs.github.com/en/get-started>

7. GitHub Hello World

A short practical exercise covering:

Repository → Branch → Change → Commit → Pull Request → Merge

This is particularly useful after students become comfortable with the Session 2 local Git workflow.

GitHub Hello World Tutorial:

<https://docs.github.com/en/get-started/using-github/hello-world>

B. Stage 2 — Front-End Craft

Sessions 3–5: HTML, CSS, Responsive Design, Accessibility, and Bootstrap

8. MDN — HTML Learning Modules

Recommended topics:

- HTML document structure
- Headings and paragraphs
- Links
- Images
- Lists
- Tables

- Forms
- Buttons
- Semantic elements
- Debugging HTML

Use MDN as the primary syntax reference when you encounter an unfamiliar HTML element or attribute.

MDN HTML Learning Path:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content

9. web.dev — Learn HTML

This is an excellent companion to MDN because it combines semantic markup with modern web-development practices.

Recommended areas:

- Document structure
- Semantic HTML
- Links
- Forms
- Images
- Accessibility
- Focus
- Content organization

web.dev Learn HTML:

<https://web.dev/learn/html>

10. MDN — CSS Learning Modules

Use this for:

- Selectors
- Cascade
- Specificity
- Inheritance
- Box model
- Units
- Flexbox

- Grid
- Responsive design
- Media queries
- Debugging CSS

MDN Core Web Development Modules:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core

11. web.dev — Learn CSS

Recommended after Sessions 4 and 5 for a deeper understanding of modern CSS.

Focus on:

- Cascade
- Specificity
- Box model
- Layout
- Flexbox
- Grid
- Responsive design
- Logical properties
- Accessibility
- Z-index and stacking

web.dev Learn CSS:

<https://web.dev/learn/css>

12. Bootstrap Documentation

Use the official Bootstrap documentation instead of relying only on copied component examples.

Recommended areas:

- Containers
- Grid
- Breakpoints
- Spacing utilities
- Flex utilities
- Typography

- Navbar
- Cards
- Forms
- Buttons
- Responsive utilities

Bootstrap Documentation:

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>

13. W3C Web Accessibility Initiative Tutorials

Accessibility should be learned alongside HTML and CSS rather than treated as an optional topic at the end of a project.

Recommended tutorials include:

- Page structure
- Menus
- Images
- Tables
- Forms
- Carousels
- Accessible components

W3C WAI Accessibility Tutorials

<https://www.w3.org/WAI/tutorials/>

C. Stage 3 — JavaScript Essentials

Sessions 6–8: JavaScript Fundamentals, Arrays, Objects, DOM, Events, and JSON

14. MDN — JavaScript Learning Modules

Use MDN as the primary reference for JavaScript language fundamentals.

Recommended subjects:

- Variables
- Data types
- Operators
- Conditionals
- Loops

- Functions
- Scope
- Arrays
- Objects
- Classes
- Promises
- Async/await
- Error handling

MDN JavaScript Learning Path:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting

15. web.dev — Learn JavaScript

This provides another structured path through modern JavaScript fundamentals and is useful for participants who benefit from seeing concepts explained in a different way.

web.dev Learn JavaScript:

<https://web.dev/learn/javascript>

16. MDN JavaScript Reference

Once you understand the basics, the MDN JavaScript Reference becomes your lookup tool.

Use it when you need to check:

- Array methods
- String methods
- Object methods
- Date
- Math
- Promise
- Error
- JSON
- Operators

MDN JavaScript Reference:

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>

17. MDN DOM Documentation

Recommended after Session 8.

Study:

- `querySelector()`
- `querySelectorAll()`
- `createElement()`
- `textContent`
- `classList`
- Event listeners
- Forms
- Browser events

MDN DOM Documentation:

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

18. MDN Fetch API

Return to this resource during both the JavaScript and React API-integration stages.

Study:

- `fetch()`
- Responses
- JSON responses
- HTTP errors
- Request options
- POST requests
- Headers
- Asynchronous programming

MDN Fetch API:

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

19. JavaScript.info

Optional supplementary reading

This resource offers detailed explanations and exercises covering the JavaScript language, browser APIs, DOM, events, promises, and asynchronous programming.

Useful when the bootcamp explanation feels too fast and you want another explanation before proceeding.

JavaScript.info:

<https://javascript.info/>

D. Stage 4 — Back-End Engineering

Sessions 9–11: Node.js, Express, MongoDB, Mongoose, REST, and Postman

20. Node.js — Official Learning Documentation

Start here before depending on random Node tutorials.

Recommended subjects:

- Introduction to Node.js
- Running scripts
- npm
- Environment variables
- Asynchronous programming
- HTTP
- File system
- Event loop
- Error handling

Node.js Learn:

<https://nodejs.org/en/learn>

21. npm Documentation

Use this to understand:

- package.json
- Dependencies
- Development dependencies
- Scripts
- Package installation
- Versioning
- npx
- npm CLI commands

npm Documentation:
<https://docs.npmjs.com/>

22. Express Documentation

Recommended sequence:

1. Installing Express
2. Hello World
3. Basic Routing
4. Middleware
5. Routing
6. Error Handling
7. Static Files

Express Documentation:
<https://expressjs.com/>

The official routing guide is especially useful while building Capstone 2.

Express Basic Routing:
<https://expressjs.com/en/starter/basic-routing/>

23. MongoDB Documentation

Begin with:

- Documents
- Collections
- CRUD
- Querying
- Data modeling
- Indexes

MongoDB stores records as documents containing field/value pairs, with documents organized into collections.

MongoDB Manual:
<https://www.mongodb.com/docs/manual/>

24. MongoDB Node.js Driver Documentation

Even though the bootcamp uses Mongoose, understanding the official MongoDB driver helps clarify what Mongoose is abstracting.

Recommended topics:

- Connecting to MongoDB
- Databases and collections
- CRUD operations
- Queries
- Indexes

MongoDB Node.js Driver Documentation:

<https://www.mongodb.com/docs/drivers/node/current/>

25. Mongoose Documentation

Recommended topics:

- Schemas
- SchemaTypes
- Models
- Documents
- Queries
- Validation
- Middleware
- Populate
- Timestamps

Mongoose's official guides organize these concepts into dedicated sections for schemas, models, validation, queries, middleware, and related data-modeling topics.

Mongoose Guides:

<https://mongoosejs.com/docs/guides.html>

26. Postman Learning Center

Use this while developing Capstone 2.

Recommended practice:

- Send GET requests

- Send POST/PATCH/DELETE requests
- Configure JSON request bodies
- Inspect status codes
- Use headers
- Save requests into collections
- Create environments
- Add simple tests
- Document API requests

Postman's official documentation covers sending requests, organizing them into collections, API testing, environments, and broader API-development workflows.

Postman Documentation:
<https://learning.postman.com/>

27. MDN — HTTP Request Methods

Keep this open when deciding whether an API operation should use GET, POST, PUT, PATCH, or DELETE.

MDN HTTP Request Methods:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

28. OWASP REST Security Cheat Sheet

Recommended after students understand basic APIs.

Use this as an introduction to security concerns that are often missing from beginner REST tutorials.

Topics include:

- HTTPS
- Access control
- Authentication
- JWT considerations
- Input validation
- HTTP methods
- Status codes
- Sensitive data

OWASP REST Security Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

29. OWASP Authentication Cheat Sheet

Recommended before adding authentication to independent projects.

OWASP Authentication Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

E. Stage 5 — React Full-Stack Development

Sessions 12–13: React, Routing, Forms, Effects, and API Integration

30. React — Official Learn Documentation

This should be the **primary React reading**.

Recommended order:

- Quick Start
- Describing the UI
- Adding Interactivity
- Managing State
- Escape Hatches

Important topics:

- Components
- JSX
- Props
- Events
- State
- Lists and keys
- Forms
- Lifting state
- Effects
- Refs

The official React Quick Start focuses on components, markup, data display, conditions, lists, events, and updating the UI.

React Learn:

<https://react.dev/learn>

31. React — Thinking in React

This is one of the most useful readings after students learn components and state.

It teaches how to:

- Break interfaces into components
- Identify static vs. changing data
- Determine what should become state
- Determine where state belongs
- Build data flow systematically

Thinking in React:

<https://react.dev/learn/thinking-in-react>

32. React — Synchronizing with Effects

Read this before using `useEffect` heavily.

The purpose is to understand when an effect is appropriate and when normal rendering or event logic should be used instead.

React Synchronizing with Effects:

<https://react.dev/learn/synchronizing-with-effects>

33. Vite Documentation

Vite is the development/build tool used for the React exercises.

Recommended topics:

- Project creation
- Dev server
- Scripts
- Environment variables
- Static assets
- Production builds

The current Vite guide documents the dev server and production build workflow and includes React project templates.

Vite Getting Started Guide:

<https://vite.dev/guide/>

34. React Router Documentation

Use this for:

- BrowserRouter
- Routes
- Links
- Dynamic route parameters
- Navigation
- Nested routing
- Data-loading patterns when progressing beyond the bootcamp

The current React Router documentation supports declarative, data, and framework modes, so students should follow the mode selected by the instructor rather than mixing examples from different modes.

React Router Documentation:

<https://reactrouter.com/>

F. Stage 6 — Testing, Deployment, and Capstone Completion

Session 14

35. Chrome DevTools Documentation

Use this when debugging:

- Console errors
- Network requests
- DOM/CSS
- Storage
- Performance
- Responsive layouts

Chrome DevTools Documentation:

<https://developer.chrome.com/docs/devtools/>

36. MDN — Testing

Recommended for continued learning after the bootcamp.

Topics include:

- Testing strategies
- Browser compatibility
- HTML/CSS issues
- Feature detection
- Automated testing

MDN Testing Modules:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Testing

37. Vite — Production Builds

Useful when preparing React applications for deployment.

Vite Building for Production:

<https://vite.dev/guide/build>

38. GitHub Documentation — Deployment and Automation

After students are comfortable with manual deployments, explore GitHub Actions and automated deployment workflows.

GitHub Actions Documentation:

<https://docs.github.com/en/actions>

G. Stage 7 — Career Launch

Session 15

39. MDN — Soft Skills for Web Developers

This is useful for career shifters because becoming employable involves more than syntax.

Recommended subjects include:

- Research
- Problem solving
- Collaboration

- Workflow
- Communication
- Learning strategies
- Finding a job

MDN Web Development Learning Area:

https://developer.mozilla.org/en-US/docs/Learn_web_development

40. GitHub — Profile and Repository Documentation

Review this before polishing your public GitHub presence.

Focus on:

- Profile README
- Pinning repositories
- Repository descriptions
- README formatting
- Contribution visibility
- Account security

GitHub Account and Profile Documentation:

<https://docs.github.com/en/account-and-profile>

H. Optional Books for Continued Study

Area	Recommended Book	Best For
HTML/CSS	<i>Learning Web Design</i> — Jennifer Niederst Robbins	Beginners who prefer a textbook-style introduction
JavaScript	<i>Eloquent JavaScript</i> — Marijn Haverbeke	Deeper JavaScript reasoning and exercises
JavaScript	<i>JavaScript: The Definitive Guide</i> — David Flanagan	Long-term JavaScript reference
Git	<i>Pro Git</i> — Scott Chacon & Ben Straub	Git fundamentals through advanced workflows
Software craftsmanship	<i>The Pragmatic Programmer</i> — David Thomas & Andrew Hunt	Professional development habits
Code quality	<i>Clean Code</i> — Robert C. Martin	Discussion of readability and maintainability; apply ideas critically rather than as rigid rules
System design	<i>Designing Data-Intensive Applications</i> — Martin Kleppmann	Advanced post-bootcamp reading
Web usability	<i>Don't Make Me Think</i> — Steve Krug	UX and usability fundamentals

For participants in a 60-hour bootcamp, these books should be considered **post-session or post-bootcamp enrichment**, not required reading.

I. Recommended Reading by Learner Type

Learner	Start With	Then Move To
Complete Beginner	MDN Getting Started → HTML → CSS → JavaScript	Git/GitHub → Node → React
Career Shifter	MDN Fundamentals + Git/GitHub	Projects → README → APIs → React → interview preparation
Refresher	Session checkpoints + MDN reference	Official framework/library documentation

Learner	Start With	Then Move To
Front-End Focus	HTML → CSS → accessibility → JavaScript → React	Performance, testing, design systems
Back-End Focus	JavaScript → HTTP → Node → Express → MongoDB	Authentication, testing, security
Full-Stack Focus	Follow the complete bootcamp sequence	Testing, architecture, security, deployment

J. Recommended Weekly Reading Pattern

Participants do not need hours of extra reading every night. A sustainable pattern is:

Before class — 15 to 20 minutes

Read the introductory documentation for the next topic.

After class — 20 to 30 minutes

Review concepts that caused difficulty.

Weekend / open practice — 60 to 120 minutes

Rebuild one exercise without copying the instructor solution.

When debugging

Search the official documentation for the exact function, method, property, HTTP status, or error message involved.

K. How to Read Technical Documentation

Do not approach documentation like a novel.

Use this process:

1. Define the question.

“What does `Array.filter()` return?”

2. Search the exact concept.

3. Read the description first.

4. Study the syntax.

5. Run the smallest example.

6. Modify the example.

7. Apply it to your own project.

8. Write one sentence explaining what you learned.

A developer's goal is not to memorize the documentation. The goal is to become capable of **finding, understanding, testing, and applying technical information independently.**

L. Source-Priority Guide

When looking for an answer, use resources roughly in this order:

1. Official documentation

MDN, React, Node.js, Express, MongoDB, Mongoose, Git, GitHub, Postman, Bootstrap, Vite.

2. Standards and security organizations

W3C/WAI and OWASP.

3. Well-established educational resources

web.dev, reputable books, structured learning platforms.

4. Community explanations

Stack Overflow, GitHub Discussions, developer blogs, videos.

5. AI coding assistants

Useful for explanation, brainstorming, debugging assistance, and reviewing ideas—but always verify technical claims against documentation and working tests.

The farther you move away from primary documentation, the more important verification becomes.

M. Post-Bootcamp Reading Roadmap

Month 1 — Reinforce

Focus on:

HTML → CSS → JavaScript → Git

Rebuild small projects without frameworks.

Month 2 — Strengthen Full Stack

Focus on:

Node → Express → MongoDB → REST → React

Extend one of the capstones instead of continuously starting new projects.

Month 3 — Professional Development

Study:

- Testing
- Authentication
- Application security
- TypeScript
- Accessibility
- Performance
- CI/CD
- Deployment
- Open-source collaboration

After these foundations are stable, learners can explore additional technologies such as TypeScript, Next.js, SQL/PostgreSQL, automated testing frameworks, Docker, cloud platforms, and more advanced system design.

The objective is **not to collect technologies**. The objective is to deepen the ability to design, build, test, debug, document, deploy, and explain working software.

***You do not become a full-stack developer by finishing slides.
You become one by repeatedly building, debugging, explaining, and improving
working software.***